

基础层：

1: 梯度爆炸和梯度消失

2: 训练大模型显存的占用

3: model size, batch size, iteration, epoch

$\text{Iteration} = \text{model size} / \text{batch size} * \text{epoch}$

4: attention 计算公式详细版 (QKV 各自的含义和计算原则)

5: ReLU、LeakyReLU、ELU、GELU, 残差连接 (ResNet), BatchNorm

6: 混合精度训练 (FP16、FP32、BF16)

前向和反向传播中, 某些操作必须用 FP32 (如 softmax, loss 计算), 否则会精度下溢

1. 前向: 权重 FP16 $\rightarrow$ 计算 $\rightarrow$ 输出 FP16 (但 Softmax 内部转 FP32)

2. Loss: 用 FP32 计算

3. 反向: 梯度用 FP16 (但 Loss Scaling 放大后再转回)

4. 参数更新: 梯度转 FP32 $\rightarrow$ 更新 FP32 的主权重 $\rightarrow$ 转回 FP16

Conclusion: “FP16 计算, FP32 更新, 关键时刻切 FP32 防止下溢”

7: 显存计算 (模型参数、梯度、优化器 (Adam SGD)、激活值):

参数占 2P (BF16/FP16)

梯度也是 2P

Adam 要 8P (两份 FP32)

激活值 看 batch

8: 分布式并行: Data、Model、Pipeline Parallelism

DP (数据并行): 每张卡做不同的题 (每张 GPU 都有完整的模型, 但是处理不同的数据) (类比学生高考)

MP (默写并行): 把模型分割成不同 layer, 分别放到不同的 GPU 上面 (类比流水线生产任务)

PP (流水线并行) ——MP 的一种特殊形式

MP 模型被切分到不同 GPU 大类，包括切层、切算子

PP 按层切分，串行执行 MP 的一种具体形式

PP = 按层切 (Layer-wise)

MP = 更细粒度切 (如切 attention 头、切矩阵)

并行方式	每张卡存什么	每张卡算什么	通信模式	什么时候用
------	--------	--------	------	-------

DP	完整的模型	不同的数据	梯度同步	模型能装进单卡，想加速
-----------	-------	-------	------	-------------

PP	模型的一部分 (连续几层)	同一份数据的不同层	层间传激活值	模型装不进单卡
-----------	------------------	-----------	--------	---------

MP	模型的一小片 (如半层)	同一层的不同部分	密集通信	单层装不进单卡
-----------	-----------------	----------	------	---------

Pipeline parallelism 将 GPU 平均分配，好处：减少空闲等待、每张卡计算量均衡

坏处：存在流水线气泡（前几张卡算完还要等后面的）、跨卡通信有开销（层与层之间要传递激活值和梯度）、微调 batch size 大小影响效率。

To 减少气泡：用 micro-batch，把一个小 batch 再切碎，流水线填满。

9：梯度累积：线性求和

10：KV Cache：

第一次生成 "我" → 计算出 "我" 的 K、V

第二次生成 "爱" → 只算 "爱" 的 K、V，前面"我"的 K、V 直接复用

第三次生成 "你" → 只算 "你" 的 K、V，前面"我""爱"的 K、V 直接复用

Conclusion：把已经算过的 K、V 缓存起来，避免重复计算。**推理速度提升 10 倍以上**

Q: KV Cache 在训练时能用吗？

A: 不能。训练时序列是已知的，不需要自回归生成。KV Cache **只在推理时用**。

11：MoE (mixture of experts)：顾名思义，混合专家模型，不要用一个巨大的 FFN 层

处理所有 token，而是用**多个“专家”**，每个 token 只激活其中几个专家。

Mixtral 8×7B：总参数 47B，但每个 token 只激活 2 个专家 → 推理时只算 ≈13B，**速度接近 13B 模型，能力接近 47B 模型**

12：Perplexity（困惑度）：计算交叉熵损失（模型对下一个 token 的平均猜测范围有多大”）交叉熵损失越低，perplexity 越低，表示模型对自己的预测约有信心，推理越准确。

Perplexity = $e^{(\text{交叉熵损失})}$ 。越低越好。人类语言的 Perplexity 通常在**几十到几百**之间。

13：量化（进行权重压缩）：把模型权重从高精度（FP16/FP32）转成低精度整数（INT8/INT4）（会有不同程度的精度损失）

精度	字节/参数	压缩比
FP16	2	1×
INT8	1	2×
INT4	0.5	4×
INT2	0.25	8×

14：MHA（multi-head attention）、GQA（grouped query attention）、MQA（multi-query attention）的区别

注意力类型	Query 头数	KV 头数	推理速度	质量
MHA	8	8	慢（基准）	最高
GQA	8	4（分组）	中	中高
MQA	8	1	最快	中

MQA 推理最快（所有 Query 共享同一组 KV，质量会降低），multi-head 最慢，GQA 折中

LLaMA 2: 用了 MHA

LLaMA 3: 用了 **GQA** (折中方案)

PaLM: 用了 MQA

15: 大模型训练的 3 个阶段及其目标:

三个阶段的数据量对比:

- PT: **几万亿** token
- SFT: **几万到几十万** 人工标注问答对
- RLHF: **几万** 人类偏好比较数据

Phase 1: PT (pre-training) 预训练:

用海量无标记文本投喂, 学习语言规律得到 base model (模型基座)

Phase2: SFT (supervised fine-tuning) 监督微调:

用人工标记的问答对, 学习对话格式, 输出 chat model (模型对话)

Phase3: RLHF (Reinforcement learning from human feedback):

用人类的反馈进行强化学习, 用上奖惩机制输出最终产品

16: RLHF (reinforcement learning from human feedback):

从人类的反馈中强化学习: step1: 收集比较数据 (人工给模型的答案打分) step2: 训练奖惩模型 (告诉模型人类偏好什么) step3: 用 PPO 强化学习 (根据奖惩模型优化 SFT 模型)

Tips: Step 3 中, 模型会自己生成答案, 奖励模型打分, 然后反向传播优化。这就是“强化学习”的部分。

PPO (Proximal Policy Optimization) 是一种强化学习算法。模型通过**试错** (自己生成答案) + **奖励信号** (奖励模型打分) 来学习, 这就是典型的 RL 范式。

17: 高效微调, QLoRA (quantization low rank adaptability):

把模型基座量化到 4-bit (显存节约 4 倍), 在此基础上做 LoRA 微调, 微调时只更新 LoRA 参数, 基座模型保持 4-bit。

效果: 用 24GB 显存就可以微调 33B 模型 (四两拨千斤)

QLoRA 论文标题: "QLoRA: Efficient Finetuning of Quantized LLMs"

效果：用 24GB 显存（单张 4090）可以微调 **33B** 模型。以前需要多张 A100。

18：Flash Attention：

问题	标准 Attention	FlashAttention
显存复杂度	$O(N^2)$	$O(N)$
中间矩阵	存整个 $N \times N$ 矩阵	分块计算，不存
IO 次数	多次读写 HBM	利用 SRAM（快 20 倍）

不把整个注意力矩阵写回显存，而是分块计算、分块 softmax、分块累加。**速度提升 2-4 倍，显存不爆炸。**

版本	核心改进
FA1	分块 + 在线 softmax
FA2	减少非矩阵乘运算，利用 Tensor Core
FA3	异步加载 + 更激进的并行

19：RoPE（Rotary Position Embedding）旋转位置编码：

传统正余弦位置编码：把位置信息直接加到词向量上面。

RoPE：把位置信息通过旋转矩阵‘乘’到 Q 和 K 上面

两个 token 的相对距离，通过旋转角度的差来表示

为什么 RoPE 更好：

-  能处理**超长序列**（传统编码外推能力差）
-  相对位置自然编码
-  LLaMA 用它，上下文可以从 4K 扩展到 128K

为什么能处理超长序列？

RoPE 的相对位置编码，让模型看到长度是**训练时 N 倍**的序列时，仍然能理解相对距离。LLaMA 3 用 RoPE + 位置插值，从 8K 扩展到了 **128K**。

进阶层

1：采样策略

Top-K 采样：从概率最高的 K 个 token 中随机采样，忽略其余 token。

和 **Temperature** 的关系：

- Temperature 控制概率分布的**平滑程度**
- Top-K 控制候选词的**数量**
- 通常**一起用**

Top-P 采样 (nucleus sampling)：从概率累计和达到 P 的最小 token 集合中采样

方法	规则	优点	缺点
Top-K	固定取 K 个	简单	有时 K 个都不靠谱，有时超过 K 个都靠谱
Top-P	动态取概率和达 P 的	自适应，更灵活	稍有计算开销

实际用法：通常 **Top-P 优先于 Top-K**，或者两者结合。

想要的效果	Top-K	Top-P	Temperature
非常保守 （几乎总是选最高概率）	K=1	P=0（实际 P=0 不会用，通常 P 很小如 0.1）	T→0
非常随机 （创意输出）	K 大（如 100）	P=1.0	T>1

2：集束搜集 (Beam Search)：每次保留 Beam Size 个最优候选序列，而不是只保留唯一一个。

方法	适用场景	特点
Temperature + Top-P	对话、创意写作	多样性好，可能“发散”
Beam Search	翻译、摘要、代码生成	确定性高，最可能输出“最优”

Q: Beam Search 的 Beam Size 越大越好吗？
A: 不是。越大越慢，而且容易产生**重复**或**平庸**的结果。

任务	推荐方法	原因
翻译、摘要、代码生成	Beam Search	有相对确定的“正确答案”
对话、故事创作、头脑风暴	Temperature + Top-P	需要多样性和创意
数学推理、事实问答	贪心 (Temperature=0)	需要精确，不要随机

3：长文本与推理高级技术：

位置插值（position interpolation）：当模型训练时最大长度是 4K，你想让它处理 8K 的文本，就把位置索引**压缩一半**。

- RoPE（你已经知道）可以处理连续的位置值
- 压缩后，模型看到的“相对角度”变小，更容易适应

NTK-aware 插值：位置插值的改进版，**对不同频率的维度用不同的压缩率**。

- RoPE 中，不同维度有不同的“旋转频率”
- 高频维度（变化快）对短距离敏感
- 低频维度（变化慢）对长距离敏感

NTK 的策略：高频维度少压缩（保持局部精度），低频维度多压缩（扩展长距离）

位置插值（PI）	所有维度统一压缩	简单，但短距离精度可能下降
NTK-aware	高频少压、低频多压	更好保持短距离性能

YaRN（Yet another RoPE extension）：NTK-aware + 温度缩放，目前**最主流的长文本**

扩展方法。

- 1. **NTK-aware**: 不同维度不同压缩率
- 2. **温度缩放**: 加一个额外的缩放因子 (微调后效果更好)
- 3. **少量微调**: 用几百步长文本数据微调即可

26	位置插值 (PI)	均匀压缩所有位置
27	NTK-aware	高频少压, 低频多压
28	YaRN	NTK + 温度缩放 = 最好用

4: 评估与可观测性

MMLU (massive multitask language understanding): 一个包含 **57 个任务** (数学、历史、法律、医学等) 的多选题数据集, 用来测模型的**综合知识水平**

为什么重要: MMLU 是**大模型必争的榜单**, 几乎所有模型发布都会报 MMLU 分数。

MT-Bench (multi-turn bench): 测试模型**多轮对话能力**的基准, 让模型和人类 / Claude 玩“角色扮演”。

Q: “对话任务”和“多轮对话任务”的区别?

A: 单轮对话 (一问一答) 测的是指令遵循能力; MT-Bench 特意测**多轮**, 看模型能否记住上文、保持角色一致。MMLU 完全不测对话, 只测**知识点**。

对比	MMLU	MT-Bench
测什么	单轮知识问答	多轮对话能力
题型	选择题	开放式问答
裁判	自动判分	Claude 打分

W&B (Weights&Biases) : AI 训练的 **可视化工具**, 用来记录 loss 曲线、显存占用、超参数等。

记录指标 loss、accuracy、perplexity

对比实验 不同学习率的效果对比

保存模型 自动存 best model

分享报告 生成可交互的链接发给团队

替代品：TensorBoard (TensorFlow 原生，但 W&B 更美观、功能更强)

TensorBoard **W&B**

本地运行 云端 + 团队协作

难对比实验 一键对比不同 run

手动存模型 自动保存最佳模型

单机查看 可生成分享链接

W&B 除了美观，最大的亮点是团队协作和实验对比功能。你可以在网页上给同事分享一个链接，对方就能看到你所有的 loss 曲线、超参数、代码版本，甚至还能在表里直接对比学习率 A vs B 的效果。TensorBoard 需要自己搭、自己存、自己发截图。

5：解码与推理服务：

Chunked prefill：当输入序列很长（比如有 10 万个 token）时，**切成小块（chunk）** 分别计算 prefill，避免一次性占满显存。

- Prefill 阶段要计算所有输入 token 之间的注意力，复杂度 $O(N^2)$
- 输入 10 万 token → 注意力矩阵有 100 亿个元素（显存直接爆炸）

Prompt Caching：**缓存**反复出现的系统提示词（如“你是一名专业医生”）的 K、V，不同用户会话复用

不缓存：每次都要给 500 token 做 prefill（慢、贵）

缓存：第一次 prefill 后存下 K、V，后面直接复用

Prefix Caching（vLLM 实现）：vLLM 中的**自动 Prompt 缓存**，只要前缀相同就复用。

Prompt Caching	Prefix Caching	
触发方式	手动配置系统提示词	自动 检测相同前缀
粒度	固定提示词	任意公共前缀
代表系统	商业服务	vLLM